

Name/NetID: Nathan Kariuki / nkari011

Session Date: April 28, 2026 (2 hour)

Objective:

K-hop BFS and callbacks, tested on small graphs first.

Attempts made:

First version of BFS just did a full traversal, K wasn't being used at all:

```
queue<int> q;

q.push(src);

while (!q.empty()) {

    int u = q.front(); q.pop();

    for (int i = offsets[u]; i < offsets[u+1]; i++) {

        int v = edges[i];

        if (!visited[v]) { visited[v] = true; q.push(v); }

    }

}
```

Counts were wrong any time K was less than the full graph depth. I tried pairing each node in the queue with its depth but the logic got messy and I kept getting wrong hop counts. Also the max callback was giving back 0 on empty results, spec said -1.

Solutions/Workarounds:

Redid it with two vectors for current and next frontier. Outer loop runs exactly K times so the depth limit just works:

```
vector<int> current_level = {src};

for (int hop = 0; hop < K; hop++) {

    vector<int> next_level;

    for (int u : current_level) {
```

```

for (int i = offsets[u]; i < offsets[u+1]; i++) {

    int v = edges[i];

    if (!visited[v]) {

        visited[v] = true;

        next_level.push_back(v);

        reachable.push_back(v);

    }

}

}

current_level = move(next_level);

if (current_level.empty()) break;

}

```

Fixed max to return "-1" on empty. Wrote RunTests() and checked a handful of cases, basic counts, K=0, isolated vertex, graph with a cycle. All good. Then ran queries20.txt:

Correctness: ALL PASS

Commits:

Add K-hop BFS with count and max callbacks

Add RunTests with small graph cases

Experiments/Analysis:

Mainly wanted to make sure cycles didn't break anything since the Slashdot graph is dense. ALL PASS on queries20 was good enough to move on.

Learning outcome:

Two frontier vectors is way cleaner than trying to track depth inside a queue. Didn't expect the early break on empty frontier to matter but it probably helps on queries with small reachable sets.